Lecture14: Tree3
Ensemble Techniques

STA 4724

HanQin Cai

University of Central Florida

Ensemble Techniques

- We have learned a good number of base ML classifiers/predictors so far
- Can we combine the base ones together to get a more powerful one?
- *Ensemble method* is a machine learning technique that combines several base models in order to produce one optimal classification/prediction model
- We will focus on classification problems as the example in this section

Weak and Strong Learners

- Weak learner: the base classifiers that provide us with a result that is slightly better than random guessing
  - Expectation of random guess is 0.5
  - Weak learner should have an expectation $\geq 0.5 + \varepsilon$ where ε is a small positive number

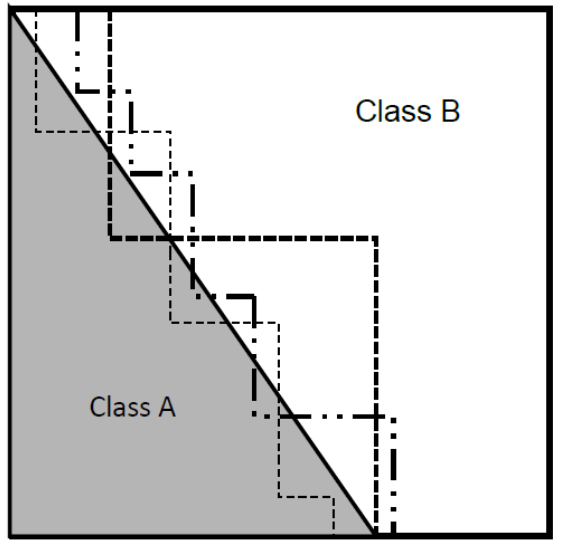
- Strong learner: provides us with a very successful classification result
  - Expectation $\gg 0.5$

- Our task is to find a way to aggregate the results provided by our weak learners and obtain a strong one

Preliminary

- The weak learners should have low bias

- The weak learners should be uncorrelated
  - Their classification errors must occur on different training records



Decision boundaries provided by several *uncorrelated* base weak learners

- A decision tree is a classic example of base classifiers that satisfy these conditions
  - Another advantage: the number of classification labels (number of classes being involved) is flexible in growing our tree

Why “Uncorrelated” Important?

- Consider 10 new data entries whose true labels are all A.
- Use a simple majority vote as our “ensemble”

Classifier	Predicted Class									
1 (Accuracy 80%)	A	A	A	A	A	B	A	B	A	A
2 (Accuracy 60%)	A	B	A	B	B	A	B	A	A	A
3 (Accuracy 70%)	B	A	A	A	A	A	B	A	A	B
Ensemble	A	A	A	A	A	A	B	A	A	A

Three classifiers together boost the accuracy to 90%

Classifier	Predicted Class									
1 (Accuracy 80%)	A	A	A	A	A	B	A	B	A	A
2 (Accuracy 70%)	B	A	A	A	A	A	B	A	A	B
3 (Accuracy 70%)	B	A	A	A	A	A	B	A	A	B
Ensemble	B	A	A	A	A	A	B	A	A	B

Accuracy is still 70%, no improvement at all

Method 1: Bagging

- ***Bagging*** stands for bootstrap aggregation

- Given an initial training dataset with *N* entries, in bagging we create multiple training datasets of size *N*, by sampling uniformly with replacement
  - Each of the training dataset is called a bootstrap
  - In each bootstrap, some data entries may be picked more than once and some may not appear at all

- We build classifiers on each bootstrap sample and take a majority vote across the classifiers

Method 1: Bagging

- Research has shown bagging is an effective methodology on learning algorithms where small changes in training set have large impacts in resulting predictions
  - These algorithms are called “unstable”
  - Some typical examples: *decision tree* or *neural network* (have you heard of “adversarial attacks”?)

- The increased accuracy from bagging comes from the reduction in the variance of the individual classifier
  - If the classifiers are stable (the error comes mainly from bias), then bagging may not be effective

- Since we are resampling the data with replacement, bagging does not focus on particular data instances of the training data. This means that we are less prone to overfitting

Method 2: Boosting

- In bagging, no specific data entries get any preferential treatment
- If we do give special treatment, which entries should get more attention?
- ***Boosting*** is an iterative methodology that adapts the sampling of the data in order to concentrate on entries that have been misclassified in previous iterations

Method 2: Boosting

- In the initial iteration, we start with a uniform distribution assigning equal weights to our *N* records
- At the end of the first round, we change the weights to emphasize those entries that were misclassified
- Repeat the process, boosting produces a series of classifiers with inputs based on the outcome of the previous classifiers
- The final prediction of the series is computed by a weighted vote, depending on the training errors of the individual base classifiers
- The non-uniform sampling (started from 2^{nd} iteration) is done by with replacement, where the previously misclassified entries are given more weights
  - Forcing the ensemble to correct for these mistakes

Method 2: Boosting

- A popular implementation of boosting is the ***Ada-Boost*** algorithm, a.k.a. adaptive boosting

- Ada-Boost is a fast algorithm with no parameters to tune, except for the number of iterations

- However, if the base classifiers are complex, Ada-Boost may lead to overfitting

- Ada-Boost may be susceptible to noise.

Method 3: Random Forests

- A single tree provides a good shade, but the canopy of a group of trees is hard to beat

- If we grow a group of various decision trees as our base classifiers, and enable their growth to use a random effect we end up with a ***random forest***
  - A modified bagging for decision trees

- The major difference: On top of growing each tree using each bootstrap, each node of the tree is split among a subset of randomly chosen features
  - Extra layer of randomness
  - Robust to overfitting

- This methodology is known as *Forest-RI* or *Random Input selection*.

Method 3: Random Forests

- A random forest algorithm has only two parameters:
  a. The number of features in the random subset at each node
  b. The number of trees in the forest

- In cases where there is a small number of features, it is possible to create new features by taking random linear combinations of the existing ones
  - This process is called Forest-RC or Random Combinations

- Running a random forest algorithm is fast, having the advantage of being robust against unbalanced or missing data

- However, with datasets that are particularly noisy, they tend to overfit

Method 4: Stacking and Blending

- Stacking and Blending Generalization: Techniques to combine multiple base classifiers of **different** kinds

- Let’s see an example of stacking

Stacking Example

- **Step 1**: We have Train Data and Test Data. Assume we are using 4-fold cross validation to train base models.

train_data

	x_0	x_1	x_2	x_3	y
fold_1	0.94	0.27	0.80	0.34	1
	0.02	0.22	0.17	0.84	0
fold_2	0.83	0.11	0.23	0.42	1
	0.74	0.26	0.03	0.41	0
fold_3	0.08	0.29	0.76	0.37	0
	0.71	0.76	0.43	0.95	1
fold_4	0.08	0.71	0.97	0.04	0
	0.84	0.97	0.89	0.05	1

test_data

x_0	x_1	x_2	x_3	y
0.74	0.17	0.820	0.31	1
0.04	0.27	0.13	0.80	0
0.87	0.10	0.24	0.39	1

Stacking Example

- **Step 2**: Let's say Model 1 is a decision tree, which is trained by 4-fold cross-validation. The trained model will be used to predict Test Data, denoted **pred_m1**.

test_data

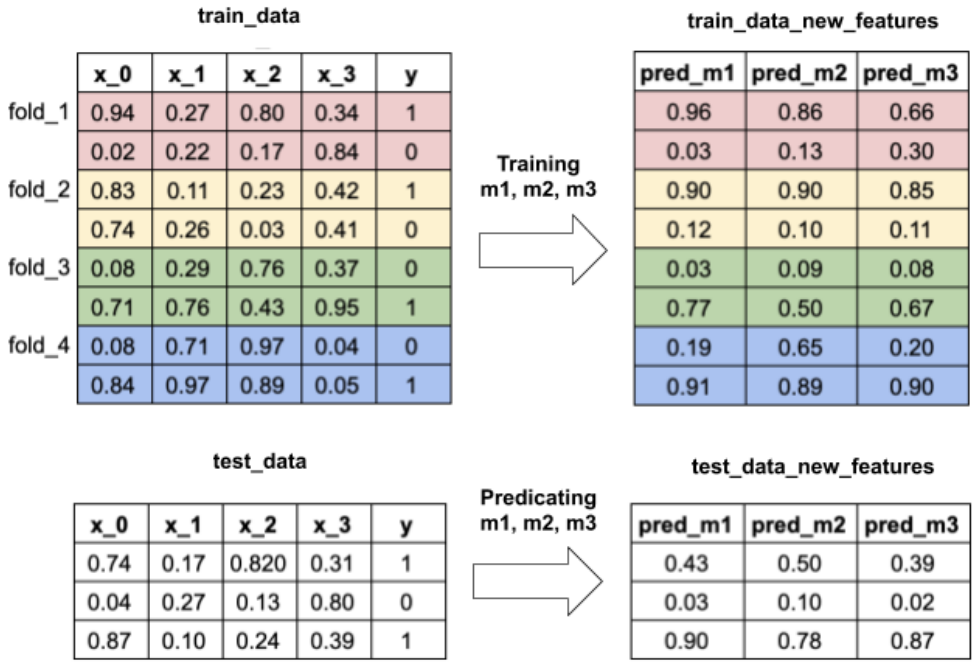
x_0	x_1	x_2	x_3	y
0.74	0.17	0.820	0.31	1
0.04	0.27	0.13	0.80	0
0.87	0.10	0.24	0.39	1

new_features

pred_m1
0.43
0.03
0.90

Stacking Example

- **Step 3**: Do the same training for Models 2 and 3, say kNN and Naïve Bayes. These will give both train_data and test_data two more features from the predictions, namely **pred_m2** and **pred_m3**.



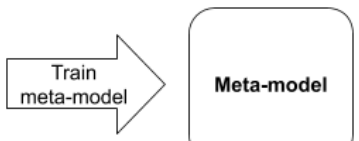
- The newly added features are called ***meta-features***

Stacking Example

- **Step 4**: Now, to train the ***meta-model*** (let's say a logistic regression), we use only the newly added features from the base models, which are [pred_m1, pred_m2, pred_m3].

New train_data for meta-model

pred_m1	pred_m2	pred_m3	y
0.96	0.86	0.66	1
0.03	0.13	0.30	0
0.90	0.90	0.85	1
0.12	0.10	0.11	0
0.03	0.09	0.08	0
0.77	0.50	0.67	1
0.19	0.65	0.20	0
0.91	0.89	0.90	1

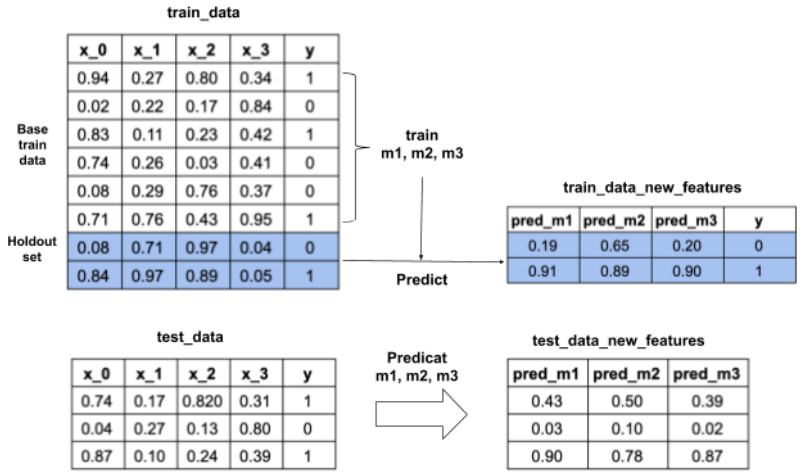


- The final prediction for test_data is given by the trained meta-model.

Blending Example

- Stacking and Blending are very similar. The difference: how to allocate the training data.

- **Step 1**: train_data is split into base_train_data and holdout_set.
- **Step 2**: Base models are fitted on base_train_data, and predictions are made on holdout_set and test_data. These will create the meta-features.



- **Step 3**: New meta-model is then fit on holdout_set with both original and meta-features.
- **Step 4**: The trained meta-model is used to make final predictions on the test data using both original and meta-features.

Method 4: Stacking and Blending

- The name blending was made popular by the winners of the famous Netflix Prize Competition, a.k.a. the Netfilx problem
  - To improve the accuracy of recommendations provided by Netflix, based on customer film preferences

- Generally speaking, blending is a simpler methodology than stacking (no cross-validation), but less data is actually used

- Note that the meta-model may overfit to the holdout set

Time to Play – Defer to Next Class

- Let’s see how to use Ensemble Techniques in **python**!